Trabajo Práctico

Ruteo de última milla

Integrantes: Gabriel Moreno y Manuel Ferrer

1. El problema y cómo lo diseñamos

La idea del trabajo es resolver el problema de una empresa de logística que tiene que hacer repartos en la ciudad. Si se calculan las rutas usando solo la distancia fija en metros, lo más probable es que en la ruta están las avenidas porque son más directas. El problema es que en la realidad el tiempo de viaje cambia mucho según la hora, ya que puede ser hora pico o una más tranquila, el día de la semana, el clima, ya que puede estar lloviendo o no y si es feriado.

Para resolverlo armamos el sistema en tres partes:

1. **Un modelo de Machine Learning** que predice cuántos segundos se tarda en recorrer cada calle según el contexto.
2. **Un grafo dirigido en Python (`networkx`)** donde los nodos son las esquinas y las aristas son las calles, con las predicciones del modelo vamos cambiando los pesos de las aristas dinámicamente y usamos dijkstra para buscar el camino más rápido.
3. **Una web (Backend en Python + Frontend en React)** donde se puede elegir el origen, destino, o varios puntos de entrega, cambiar la hora y el clima, y ver en el mapa cómo cambia la ruta.

Diseño del mapa (Grafo): Armamos una cuadrícula de 5x5 (25 esquinas y 48 tramos de calle), para que tenga sentido como una ciudad real y siempre se pueda llegar de cualquier esquina a cualquier otra sin quedar encerrado, definimos tres reglas claras:

- Hay una avenida horizontal y una vertical en el medio que son doble mano.
- Las calles comunes son de mano única y van alternando de dirección (dos calles seguidas nunca van para el mismo lado).
- Las dos calles que están a los lados de una avenida tampoco van para el mismo lado (una va al este y la otra al oeste, una al norte y la otra al sur).

2. El Dataset Sintético

Como los datasets reales de tráfico suelen tener solo el inicio y fin del viaje y no el tiempo cuadra por cuadra de nuestro grafo, armamos un script en Python (`backend/src/generate_city_and_data.py`) para generar los datos sintéticos. Generamos 5.760 viajes históricos (120 por cada tramo) usando las columnas que pide la consigna:

- `tramo_id`: el ID de la calle (`T_001` a `T_048`).
- `hora_dia`: de 0 a 23 hs.
- `dia_semana`: lunes a domingo.
- `clima`: `soleado`, `lluvia` o `tormenta`.
- `es_feriado`: `True` o `False`.
- `velocidad_promedio_historica`: velocidad típica de esa cuadra en km/h.
- `tiempo_real_viaje`: la variable a predecir (en segundos).

Reglas que usamos para generar los tiempos: Partimos del tiempo base sin tráfico (`distancia / velocidad_base`, donde las avenidas tienen velocidad base de 45–50 km/h y las calles de 35 km/h) y le aplicamos multiplicadores según el contexto:

- **Hora pico (7 a 9 hs y 17 a 20 hs en días de semana no feriados):** Las avenidas se llenan de tráfico y tardan bastante más (multiplicamos el tiempo por 2.5 a la mañana y 2.8 a la tarde), mientras que las calles barriales se traban mucho menos (x1.15 y x1.20).
- **Madrugada (0 a 5 hs) y feriados/fines de semana:** Hay menos autos y se viaja más rápido (x0.85 a x0.90).
- **Clima:** Con `lluvia` el tiempo aumenta (x1.35 en avenidas y x1.15 en calles) y con `tormenta` aumenta todavía más (x1.80 en avenidas y x1.30 en calles).
- **Ruido aleatorio:** Le sumamos una variación aleatoria normal (de ±8% aprox.) para simular cosas reales como agarrar un semáforo en rojo o un auto frenado, así el modelo no aprende una cuenta exacta sino una tendencia realista.

3. Entrenamiento del Modelo y Resultados

En `backend/src/train_model.py` preparamos los datos y entrenamos los modelos:

- Como los modelos no entienden texto (como `"lunes"` o `"lluvia"`), usamos `OneHotEncoder` para pasar las columnas categóricas a columnas de ceros y unos, y `StandardScaler` para las numéricas.
- Dividimos los datos en 70% para entrenamiento (train) y 30% para prueba (test), para tener una buena cantidad de datos de prueba que el modelo nunca haya visto.

Probamos 4 modelos y los medimos en el conjunto de test con MAE, RMSE y R^2:

Modelo	MAE (error promedio)	RMSE	R^2 Score
Regresión Lineal	7.74 seg	12.00 seg	0.2110
KNN	6.79 seg	11.15 seg	0.3187
Árbol de Decisión	3.71 seg	5.31 seg	0.8456
Random Forest (Elegido)	**2.92 seg**	**4.29 seg**	**0.8993**

Análisis de los resultados: La Regresión Lineal dio bastante mal con $R^2 = 0.21$, porque la hora del día no tiene un comportamiento lineal: el tráfico sube a las 8 de la mañana, baja al mediodía, vuelve a subir a las 6 de la tarde y baja a la noche. Una recta no puede adaptarse a esas subidas y bajadas, pero el Árbol de Decisión y sobre todo el Random Forest funcionaron muy bien con $R^2 = 0.90$ y un error promedio de menos de 3 segundos por cuadra, ya que los árboles pueden separar por rangos de horas y combinar condiciones (por ejemplo: *si es avenida + hora pico + tormenta*). Guardamos el Random Forest en `modelo_tiempo_viaje.joblib` para usarlo desde el grafo.

4. Algoritmos de Grafos e Integración con el Modelo

Toda la lógica de grafos está en `backend/src/graph_router.py`:

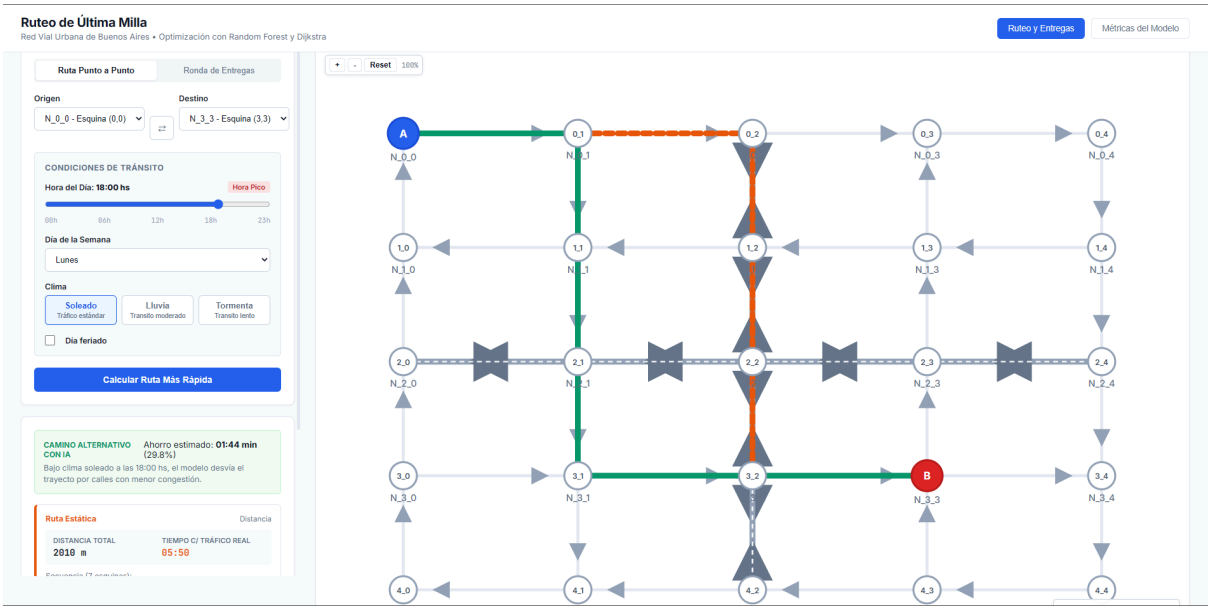
- **Integración modelo → grafo (`actualizar_pesos`):** Cuando el usuario elige en la web una hora, un día y un clima, el backend arma una tabla con los 48 tramos de la ciudad y se la pasa al modelo (`modelo.predict()`). El tiempo en segundos que predice el modelo para cada calle se guarda como el nuevo peso `tiempo_segundos` de esa arista en el grafo. Tiene sentido usar directamente esa salida como peso porque el tiempo total de un recorrido es la suma de lo que tardás en cada cuadra, y como siempre es positivo (le pusimos un mínimo de 5 segundos por seguridad), cumple con lo que necesita dijkstra.

- **Camino más rápido con dijkstra (`calcular_ruta_dijkstra` y `comparar_rutas`):** Implementamos dijkstra pasándole qué atributo queremos minimizar:

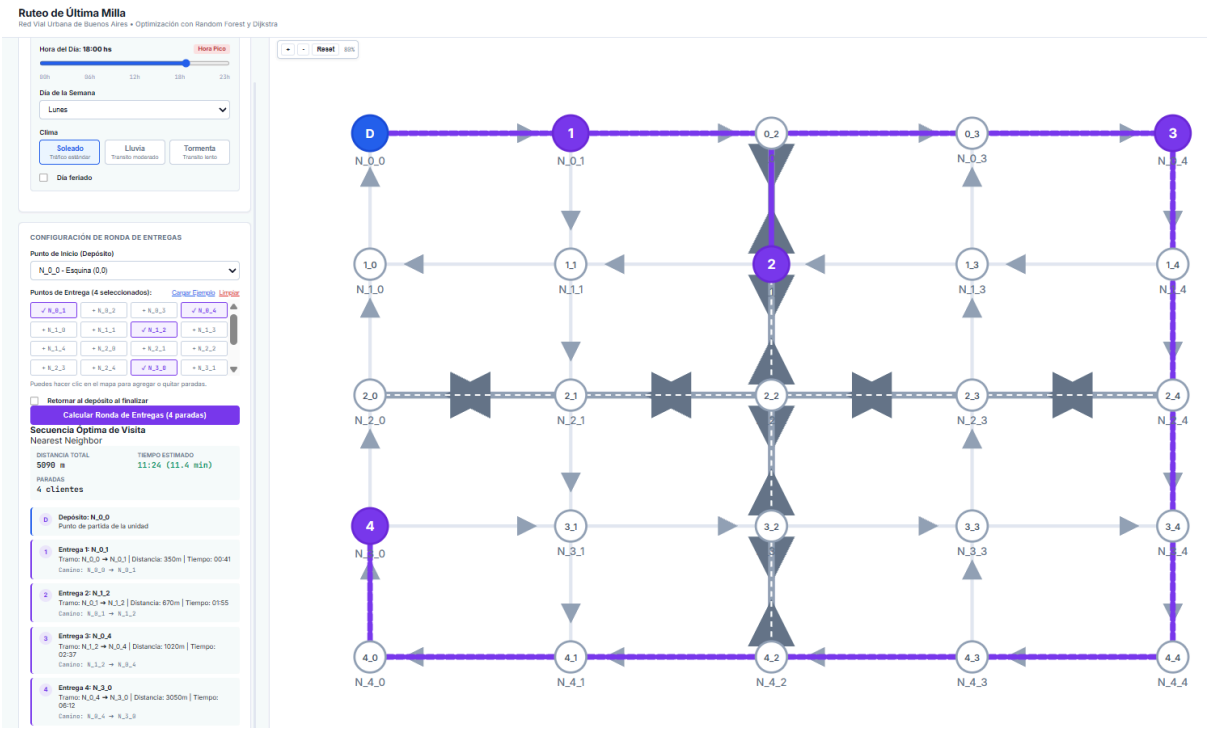
    - **Ruta Estática:** corre dijkstra usando `distancia_metros`. Como busca hacer menos metros, te manda por la avenida. Después calculamos cuánto tardaría realmente ese camino con el tráfico del momento.
    - **Ruta Dinámica:** corre dijkstra usando `tiempo_segundos` con los pesos predichos por la IA, cuando hay hora pico o tormenta, dijkstra "ve" que las aristas de la avenida tienen un costo en segundos muy alto y desvía la ruta por las calles internas.
    - *Ejemplo:* Yendo de `N_0_0` a `N_3_3` un lunes a las 18 hs con tormenta, el camino estático por avenida tarda 08:43 min, mientras que el dinámico con IA se desvía por calles barriales y tarda 05:13 min (ahorra 03:29 min, un 40% menos). También agregamos una función con `divmod` para formatear todos los tiempos en `minutos:segundos` (`MM:SS`) para que sea más fácil de leer.
- **Ronda de entregas con varios destinos (`calcular_ronda_entregas`):** Para cuando el camión tiene que salir del depósito y repartir en varios puntos, implementamos la heurística del Vecino Más Cercano (*Nearest Neighbor*) usando dijkstra, desde el punto actual calculamos con dijkstra cuánto se tarda a cada uno de los destinos pendientes, vamos al que esté más cerca en tiempo, lo tachamos de la lista y repetimos desde ahí hasta terminar (con la opción de volver al depósito al final).

5. Capturas de la Aplicación Web

- **Captura del mapa mostrando la ruta estatica (naranja) comparada con la dinamica (verde)**.



- **Captura del mapa mostrando el recorrido con varias paradas y el orden de visita.**



- **Captura de la pantalla de comparación de las métricas de los 4 modelos**.



6. Uso de Inteligencia Artificial

Parte Gabriel:

- **Herramienta usada:** Use Antigravity (con Gemini) como asistente y guía durante el desarrollo.
- **Para qué la use y para qué no:** La use para que me vaya explicando los conceptos paso a paso (como si fuera un profesor), para armar la base del script generador de datos, que me ayude tambien con la sintaxis de `scikit-learn` y `networkx`, y redactar los *Issues* de GitHub para dividirnos el trabajo entre manuel y yo. No la use como reemplazo de pensar el problema y tampoco para las reglas de cómo tenían que funcionar las calles, la división de ramas en Git y las pruebas las fui dirigiendo y corrigiendo yo.
- **Ejemplos de prompts que me sirvieron:**

1. *"Cómo tendría que ser la lógica de la función para calcular la ruta con menos peso, no me des código si no te lo pido."*
2. *"Dame unos issues para dejarle a Manu en el repositorio, así sabe exactamente qué tiene que hacer en base a la división de trabajos."*
3. *"No es mejor calcular, teniendo el tiempo en segundos y minutos, usar mod con 60 para así poder escribir 'minutos:segundos' de un camino?"*

- **Casos donde la IA se equivocó y cómo me di cuenta:**
    1. **El sentido de las calles y las avenidas:** Al principio la IA armó el grafo con dos avenidas pegadas y con calles que iban todas para el mismo lado. Cuando le pedi que alternara los sentidos, usó `r % 2 == 0` con el número de fila, lo que hacía que las dos calles a los lados de la avenida fueran para el mismo lado, y además dejó esquinas sin salida que tiraban error `404 no existe camino entre N_0_0 y N_3_3` en bucle. En otro intento, la IA directamente transformó las avenidas en calles de una sola mano, de esto me di cuenta viendo las flechas dibujadas en el mapa del frontend y viendo el error en la consola. Lo solucione parando y marcándole las tres reglas exactas (avenidas doble mano, calles seguidas alternadas y calles al lado de la avenida en distinto sentido usando un contador aparte).
- **Reflexión:** La IA me ayudó muchísimo a avanzar y terminar de entender bien el lenguaje de python, pero si no hubiéramos revisado el mapa y probado todo por consola, habrían quedado errores importantes en el grafo y en las funciones.

Parte Manuel:

- **Herramienta usada:** Use Antigravity (con Gemini y claude) como asistente durante el desarrollo.
- **Para qué la use y para qué no:** La use para que me vaya explicando los conceptos paso a paso y cuando tocaba código yo la supervisaba y testeaba para probar su correcto funcionamiento. No la use para que testee el código sin yo antes de subirlo verificarlo ni para que suba los cambios a github.
- **Ejemplos de prompts que use:**
    1. *“Antes de comenzar el proyecto asegúrate de trabajar de manera ordenada y estructurada, verificar el correcto funcionamiento del código una vez implementado y no hacer más de lo que se te pida.”*
    2. *"Asegúrate de que todo lo que se pide en el issue 1 haya sido implementado correspondientemente. Si falta implementar algo dime que es lo que falta"*
    3. *"Crea el README.md en el que debe aparecer todos los comandos para la instalación y ejecución del proyecto ordenados y una breve descripción de qué trata este proyecto y para qué sirve."*
- **Casos donde la IA se equivocó y cómo me di cuenta:**
    1. **Caso en que se equivocó:** Al principio cuando estaba haciendo el primer issue, me di cuenta de que la IA quería incluir más cosas de las que se le pedía. Las cosas que quería incluir la IA eran más variables para que el modelo a entrenar tome en cuenta durante el entrenamiento.
    Cómo me di cuenta: Me di cuenta ya que, mientras la IA estaba analizando que hacer, una de las líneas de “pensamiento” decía de agregar más

variables para el entrenamiento del modelo. Cuando lo vi detuve a la IA y le aclaré en el prompt que no debía alterar los datos de entrenamiento.

- **Reflexión:** La IA es una herramienta, y al igual que cada herramienta que existe, la persona que la usa decide de qué forma usarla y debe de entender cómo es que funciona. Sin duda la IA agiliza la corrección y creación de código pero debe de ser monitoreada y usada correctamente.

7. Limitaciones y posibles mejoras

- **Limitaciones:**
  - Para la ronda de entregas usamos la heurística del Vecino Más Cercano, que es muy rápida pero al elegir siempre el siguiente más cercano en el momento (*greedy*), no siempre da el mejor recorrido global si hubiera muchísimas paradas.
  - El mapa es una cuadrícula sintética y el tiempo se calcula solo por cuadra, sin sumar demoras extras por doblar a la izquierda o semáforos puntuales en cada esquina.
- **Posibles extensiones:**
  - Cargar el mapa real de una ciudad desde OpenStreetMap usando la librería `osmnx`.
  - Agregar una mejora tipo *2-opt* para optimizar todavía más el orden de la ronda de entregas.
  - Sumar horarios límite de entrega para cada cliente o capacidad máxima de paquetes por camioneta.